

遙測與空間資訊之分析與應用 - Week 5

國家災害防救科技中心 資訊組

台灣大學土木工程學系 教授

蘇文瑞

wraysu@ntu.edu.tw

The Captain's Intuition: The Living Map

- Week 3: Where — How far are shelters from rivers?
- Week 4: How Much — How steep is the terrain?
- Week 5: When & What-If — How hard is it raining right now?

Static Analysis (W3-W4): Potential risk.

Dynamic Monitoring (W5): The pulse of the disaster.

- The What-If Question:
- If it's sunny during class, the API returns 0mm.
- Solution: Time Machine Mode — we summon a Typhoon.

Case Study: Typhoon Fung-wong (Typhoon Fung-wong 2025)

- **East Coast — Urban Flooding (Yilan Suao):**
 - Hourly rainfall: 130.5mm (2nd highest on record at Suao station).
 - 3-hour total: 334mm (highest since Typhoon Megi 2010).
 - Severe urban flooding, schools and offices closed.
- **East Coast — Mountain Disaster (Hualien Wanrong):**
 - Nanao accumulated: 1,062mm.
 - Mataian Creek: landslide dam (40m high, 1M tons of water).
 - Wanrong, Guangfu, Fenglin — 3 townships evacuated, 356 ha flooded.
- **Course Link: Week 2 used this same creek to teach CRS importance.**

The Glass Box: History as Live Data

Problem: API returns 0mm on a sunny day — boring demo.

- **The Glass Box Strategy:**

- 1. Download historical rainfall JSON from Typhoon Fung-wong.
- 2. Mock the API response: system reads Past as Now.
- 3. Analysis logic stays exactly the same.

- **This is Mock Testing in software engineering.**

- Fire departments don't wait for typhoons to test their systems.

Your code shouldn't care if data is live or historical — the logic must be identical.

Technology Stack: CWA API

- CWA Open Data Platform: opendata.cwa.gov.tw
- O-A0002-001: Automatic Rainfall Stations (~700+ nationwide)
- API Call:
- GET /api/v1/rest/datastore/O-A0002-001?Authorization=KEY&format=JSON
- Response: `records.Station[]` with:
- StationName, GeoInfo (lat/lon), RainfallElement (hourly, 3hr, 24hr)
- Watch out: Precipitation = -998 means NO DATA, not negative rain!

Parsing JSON: Nested Dict to GeoDataFrame

- **CWA JSON is deeply nested:**
 - Station name: `Station[i]["StationName"]`
 - Coordinates: `Station[i]["GeoInfo"]["Coordinates"][0]`
- Hourly rain:
`Station[i]["RainfallElement"]["Past1hr"]["Precipitation"]`
- **Conversion pipeline:**
 1. `requests.get() → resp.json()`
 2. Loop each station → list of dicts
 3. `pd.DataFrame() → gpd.GeoDataFrame(points_from_xy)`
 4. Set CRS = EPSG:4326 (API returns GPS coordinates)

Infrastructure First: The .env Mode Switch

- **APP_MODE variable in .env:**
 - LIVE: Call CWA API for real-time data.
 - SIMULATION: Load historical JSON (Typhoon Fung-wong snapshot).
- **The Code Pattern:**
 - if APP_MODE == "LIVE": data = fetch_cwa_api(key)
 - else: data = load_json(SIMULATION_DATA)
- **Both return identical JSON structure → zero code changes downstream.**

In industry this is called **Dependency Injection** — your analysis logic never depends on a specific data source.

Folium: The Python Bridge to Leaflet.js

➤ Why Folium?

- matplotlib = static images; `.explore()` = quick preview
- Folium = full interactive maps → exports as `.html`

➤ Basic syntax:

- `m = folium.Map(location=[23.5, 121], zoom_start=7)`
- `folium.Marker([lat, lon], popup="text").add_to(m)`
- `m.save("map.html")`

`.explore()` is a draft; Folium is the final report.

CircleMarker: Encoding Data in Size & Color

➤ **Marker vs CircleMarker:**

- Marker: fixed icon, good for few points.
- CircleMarker: variable size & color, good for data visualization.

➤ **Rainfall → Color Scale:**

- < 10mm: Green (Safe)
- 10-40mm: Gold (Caution)
- 40-80mm: Orange (Warning)
- > 80mm: Red (Danger)

➤ **Rainfall → Radius: $\text{radius} = \max(5, \text{rain_mm} / 5)$**

HeatMap: Rainfall Intensity Visualization

- **Concept: Color gradient showing continuous distribution.**
 - Similar to CWA's official rainfall maps.
- **Folium HeatMap:**
 - `from folium.plugins import HeatMap`
 - `heat_data = [[lat, lon, rain] for ...]`
 - `HeatMap(heat_data).add_to(m)`
- **Limitation: Rainfall station coverage is uneven.**
 - Dense in plains, sparse in mountains → HeatMap has blind spots.

A mountain area that looks safe on HeatMap may simply have no stations.

LayerControl: Comparing Scenarios

- **Multi-layer map:**
 - Layer 1: Rainfall stations (CircleMarkers)
 - Layer 2: Rainfall HeatMap
 - Layer 3: W3-W4 Shelter risk distribution
- **LayerControl: toggle switch in the top-right corner.**
 - `folium.LayerControl().add_to(m)`

Professional GIS systems always have layer control — Folium does it in 2 lines.

Popup: Information on Click

- **Popup:** shown when user clicks a marker.
- **Tooltip:** shown when user hovers over a marker.
- **HTML in Popup — rich formatting:**
- Bold station name, colored risk level, data table.
- `popup_html = f"<b>{name}</b> <br> Rain: {rain}mm"`
- **Design principle for emergency managers:**
- First glance: risk level (big + colored).
- Second glance: key numbers (rainfall, slope).
- Third glance: recommended action.

Time Machine: Loading the Typhoon

Demo: Switch .env from LIVE to SIMULATION

Step 1: Change APP_MODE=SIMULATION in .env

Step 2: Re-run the notebook

Step 3: Watch the map transform

- LIVE mode: mostly green/yellow (sunny day).
- SIMULATION mode: eastern Taiwan turns RED (Typhoon Fung-wong peak).
- **The Shock Moment:**
 - Suao: 130.5mm/hr. Mataian Creek forming a barrier lake.
 - Your shelters — are they safe?

This is stress testing. Your ARIA system working on a sunny day means nothing.

Dynamic Risk Overlay: Rainfall x Shelter Risk

➤ Spatial overlay logic:

1. Rainfall stations GDF (4326 → convert to 3826).
2. Create 5km buffer around high-rain stations.
3. `gpd.sjoin(shelters, rain_buffers)` → affected shelters.

➤ Dynamic risk classification:

- CRITICAL: Nearby hourly rain > 80mm (regardless of original risk).
- URGENT: Nearby rain > 40mm AND `terrain_risk == HIGH`.
- WARNING: Nearby rain > 40mm OR `terrain_risk == HIGH`.
- SAFE: All others.

CRS Alignment Revisited

- CWA API gives EPSG:4326 (degrees).
- W3-W4 data is in EPSG:3826 (meters).
- Golden Rule: All layers must share CRS before spatial operations.
- Strategy:
 - Analysis (buffer, sjoin): Use EPSG:3826.
 - Visualization (Folium): Convert back to EPSG:4326.
- Most common error:
 - Forget CRS conversion → sjoin returns empty → "My map shows nothing!"

Every week we remind you about CRS alignment — because it's the **#1** error source, and it never throws an exception.

Professional GitHub: Scenario Versioning

➤ Recommended directory structure:

- project/ .env (gitignored) | .gitignore
- data/scenarios/fungwong_202511.json (historical snapshot)
- src/data_engine.py (mode switcher logic)
- ARIA_v3.ipynb | README.md

➤ Rules:

- Small historical JSON (< 1MB): commit for reproducibility.
- Never commit: .tif (DEM), .env (API keys).

Anyone who clones your repo can replay Typhoon Fung-wong — that's reproducible disaster analysis.

AI Pitfall: API Keys & Static Testing

- **Trap 1: AI hardcodes API key in source code.**
- Push to GitHub → key leaked → security incident.
- **Trap 2: AI writes wrong JSON path for CWA structure.**
- Need Glass Box verification: print each level of the JSON.
- **Trap 3: Testing with only one scenario.**
- Works for Fung-wong, breaks for the next typhoon.
- **Captain's Check:**
- Is the API key in .env? Does SIMULATION mode still work?
- How does your code handle Precipitation = -998?

UX for Emergency Managers

- **High-contrast design:**
 - Emergency: red background, white text.
 - Safe: green accents.
- **Popup information hierarchy:**
 - 1st glance (1 sec): Risk level — large, colored.
 - 2nd glance (3 sec): Key data — rainfall, slope, distance.
 - 3rd glance (5 sec): Recommended action.
- **MarkerCluster: auto-group when too many markers.**

**A commander in a typhoon has 3 seconds to read your map.
Colors and sizes must tell the story instantly.**

Learn One Pattern, Solve a Hundred Problems

➤ The Week 5 Core Pattern:

- Fetch (Live or Historical)
- → Parse JSON → GeoDataFrame
- → Spatial Overlay with Infrastructure
- → Conditional Risk Styling
- → Interactive Map (.html)

➤ This pattern applies to:

- Earthquakes, landslides, air quality, epidemics...
- Any API or historical data with spatial coordinates.

Master this pattern, and you can turn any real-time data into a disaster map.

Week 5 Reflection

A monitoring system that works in the sun is a toy. A system that survives Typhoon Fung-wong is a tool.

➤ Reflection questions:

- How many lines of code changed between LIVE and SIMULATION? (Ideal: 0)
- What if the CWA API goes down? (Fallback to latest snapshot)
- Beyond Fung-wong, what scenario would you test?

➤ Homework: ARIA v3.0

- Core: Mode Switcher + Spatial Overlay + Folium Interactive Map.
- Bonus: Gemini SDK — AI-generated emergency advice in Popups.

➤ Next Week: Week 6 — LLM-Assisted Spatial Reasoning